

Test

Test

Come verificare il comportamento del codice con test automatici semplici.

Perche scrivere test

Un test automatico controlla che una parte del programma funzioni come previsto.

Invece di provare tutto a mano ogni volta, scrivi codice che verifica altro codice.

Questo e utile quando:

- modifichi una funzione
- vuoi evitare regressioni
- lavori su programmi piu grandi

Un metodo da testare

Immagina questa classe:

```
public class Calcolatrice {  
    public static int somma(int a, int b) {  
        return a + b;  
    }  
}
```

Vogliamo controllare che `somma(2, 3)` restituisca `5`.

L'idea di test unitario

Un **test unitario** controlla una piccola unita di codice, spesso un metodo.

Con JUnit, una libreria molto usata per i test Java, potresti scrivere:

```
public class CalcolatriceTest {  
    @Test  
    void sommaDueNumeri() {  
        int risultato = Calcolatrice.somma(2, 3);  
        assertEquals(5, risultato);  
    }  
}
```

`assertEquals(5, risultato)` significa: mi aspetto che `risultato` sia uguale a `5`.

Test che passa e test che fallisce

Se il metodo è corretto:

```
return a + b;
```

il test passa.

Se per errore scrivi:

```
return a - b;
```

il test fallisce e ti segnala che il risultato ottenuto non corrisponde a quello atteso.

Cosa testare

Inizia da metodi semplici che restituiscono un valore.

Esempi buoni:

```
calcolaTotale(2.50, 4)
isMaggiorenni(20)
formattaNome("luca")
```

Sono facili da controllare perché hanno input chiari e output chiari.

Evitare test troppo grandi

Un test dovrebbe controllare una cosa principale.

Meglio tre test piccoli:

- somma numeri positivi
- somma con zero
- somma numeri negativi

che un solo test lungo che controlla tutto insieme.

Test e strumenti di build

JUnit di solito si usa dentro un progetto Maven o Gradle.

Con Maven, il comando tipico è:

```
mvn test
```

Con Gradle:

```
gradle test
```

Questi strumenti compilano il codice, eseguono i test e mostrano il risultato.

Regola pratica

All'inizio non devi testare ogni riga.

Prova a testare i metodi che:

- fanno calcoli
- controllano condizioni
- trasformano dati
- possono rompersi quando li modifichi

I test non sostituiscono il ragionamento, ma ti danno una rete di sicurezza.